1. Objective

The objective of this assignment is to develop an **HTML Structure Validator** using **LEX/FLEX** and **YACC/Bison**.

The validator should analyze a simplified HTML document and determine whether it follows the specified HTML grammar. The program should validate the structural correctness of the document and report syntax errors whenever invalid HTML constructs are encountered.

2. Learning Outcomes

After completing this assignment, students will be able to:

- Design lexical analyzers using LEX/FLEX.
- Develop recursive grammars using YACC/Bison.
- Parse hierarchical document structures.
- Understand recursive parsing of nested languages.
- Implement syntax-directed translation.
- Perform basic semantic validation.
- Generate meaningful syntax and semantic error messages.

3. Background

HTML (HyperText Markup Language) is a markup language used to create web pages.

Unlike arithmetic expressions, HTML documents consist of **nested elements**, making HTML an excellent example of a context-free language suitable for parser construction.

For example,

```
<html>
<head>
<title>Compiler Design</title>
</head>
<body>
<h1>Compiler Design</h1>
<p>Welcome to the Compiler Design Laboratory.</p>
</body>
</html>
```

The parser should verify whether the HTML document follows the defined grammar.

4. Problem Statement

Develop a program using **LEX** and **YACC** that validates the structure of a simplified HTML document.

The validator should verify

- correct document structure,
- correct nesting of HTML elements,
- balanced opening and closing tags,
- valid placement of HTML elements.

The validator is **not** required to render HTML.

5. Supported HTML Elements

The validator shall support the following HTML tags.

<html>
<head>
<title>
<body>
<h1>
<h2>
<p>

<b>
<i>

Students may extend the grammar with additional tags.

6. Sample Valid HTML

```
<html>

<head>
<title>Compiler Design</title>
</head>

<body>
<h1>Compiler Design Lab</h1>

<p>
Welcome to the Compiler Design Laboratory.
</p>

<div>
<b>Compiler</b>
<i>Design</i>
</div>

</body>
</html>
```

7. Sample Invalid HTML
Incorrect Closing Tag

```
<p>
Compiler Design
</div>
```

Incorrect Nesting

```
<b>
<i>
Compiler
</b>
</i>
```

Missing Closing Tag

```
<body>
<h1>Hello
</body>
```

Missing <body>

```
<html>
<head>
<title>Test</title>
</head>
</html>
```

8. Supported Language Features
The validator should support

- HTML document
- HEAD section
- TITLE
- BODY section
- Headings
- Paragraphs
- DIV
- Bold text
- Italic text
- Nested HTML elements
- Empty HTML elements
- Plain text

9. Language Grammar

The following grammar may be used.

```
document
        → html_document

html_document
        → HTML_OPEN
          head_section
          body_section
          HTML_CLOSE

head_section
        → HEAD_OPEN
          title_section
          HEAD_CLOSE

title_section
        → TITLE_OPEN
          TEXT_CONTENT
          TITLE_CLOSE

body_section
        → BODY_OPEN
          content
          BODY_CLOSE

content
        → content content_item
        | ε

content_item
        → TEXT_CONTENT
        | heading
        | paragraph
        | div
        | bold
        | italic

heading
        → H1_OPEN
          content
          H1_CLOSE

        | H2_OPEN
          content
          H2_CLOSE
```

paragraph
→ P_OPEN
content
P_CLOSE

div
→ DIV_OPEN
content
DIV_CLOSE

bold
→ B_OPEN
content
B_CLOSE

italic
→ I_OPEN
content
I_CLOSE

10. Lexical Analysis

The lexical analyzer should recognize the following tokens.

HTML Element	**Token**
<html>	HTML_OPEN
</html>	HTML_CLOSE
<head>	HEAD_OPEN
</head>	HEAD_CLOSE
<title>	TITLE_OPEN
</title>	TITLE_CLOSE
<body>	BODY_OPEN
</body>	BODY_CLOSE
<h1>	H1_OPEN
</h1>	H1_CLOSE
<h2>	H2_OPEN
</h2>	H2_CLOSE
	P_OPEN
	P_CLOSE
	DIV_OPEN
	DIV_CLOSE

HTML Element	Token
	B_OPEN
	B_CLOSE
	I_OPEN
	I_CLOSE
Character data	TEXT_CONTENT

Whitespace between tags may be ignored.

11. Definition of TEXT_CONTENT

TEXT_CONTENT is a lexical token returned by the scanner.
It represents **all character sequences appearing between two HTML tags**.
Examples
Compiler Design
↓
TEXT_CONTENT
Welcome to Compiler Lab
↓
TEXT_CONTENT
12345
↓
TEXT_CONTENT
The lexical analyzer should **not** include HTML tags inside the TEXT_CONTENT token.
One possible LEX pattern is
[^<\n]+
which matches every sequence of characters until the next <.

12. Syntax Analysis

The YACC parser should

- validate the complete HTML document,
- verify proper nesting of HTML elements,
- ensure matching opening and closing tags,
- detect syntax errors,
- report meaningful error messages.

Because these constraints are directly enforced by the grammar, mismatched tags and incorrect nesting are considered **syntax errors**, not semantic errors.
Example
HTML document is structurally valid.
or
Syntax Error:
Expected </p>.

13. Basic Semantic Validation (Optional)

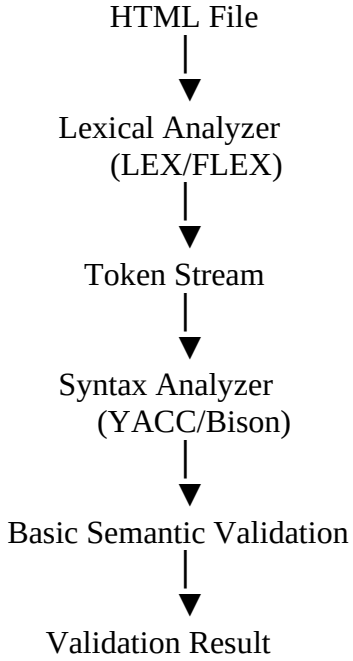
After successful parsing, students may implement additional semantic checks that cannot be conveniently enforced by the grammar.

Examples include

- Ensure only one <html> element exists.
- Ensure only one <head> section exists.
- Ensure only one <body> section exists.
- Detect duplicate id attribute values (extension).
- Validate HTML attributes (extension).

Unlike syntax errors, these checks require additional program logic and are therefore considered **semantic analysis**.

14. Suggested Program Workflow



15. Suggested Project Structure

validator.l
validator.y

input.html
output.txt

16. Suggested Development Strategy

Students are encouraged to develop the project incrementally.

Step 1
Recognize all supported HTML tags.

Step 2
Recognize TEXT_CONTENT.

Step 3
Parse a minimal HTML document.
<html>

<body>

</body>

</html>

Step 4
Add HEAD and TITLE sections.

Step 5
Support headings.

Step 6
Support paragraphs.

Step 7
Support nested DIV elements.

Step 8
Support nested formatting elements (<b>, <i>).

Step 9
Implement optional semantic validation.

17. Suggested Error Messages

Syntax Error:
Expected </body>.
Syntax Error:
Unexpected .
Syntax Error:
Expected </p>.
Semantic Error:
Multiple <body> sections are not allowed.

18. Extension Tasks

Students may implement one or more of the following features.

- Support unordered lists (<ul>, <li>).
- Support ordered lists (<ol>, <li>).
- Support tables (<table>, <tr>, <td>).
- Support hyperlinks (<a>).
- Support images (<img>).

19. Submission Requirements

Submit a single report containing:

- Problem description
- LEX source file (validator.l)
- YACC source file (validator.y)
- Sample HTML input (input.html)
- Sample output (output.txt)